

Thông tin môn học

- *Mã môn học: MUL1446*
- *Tên môn học: Lập trình Game cơ bản*
- *Tên tiếng Anh: Basic Game Programming*
- *Số tín chỉ: 03*
- *Lý thuyết: 28 tiết*
- *Bài tập: 08 tiết*
- *Thực hành: 8 tiết*
- *Tự học: 1 tiết*



LẬP TRÌNH GAME CƠ BẢN

Giảng viên: Ths. Trần Duy Hiếu

Email: hieutd@ptit.edu.vn

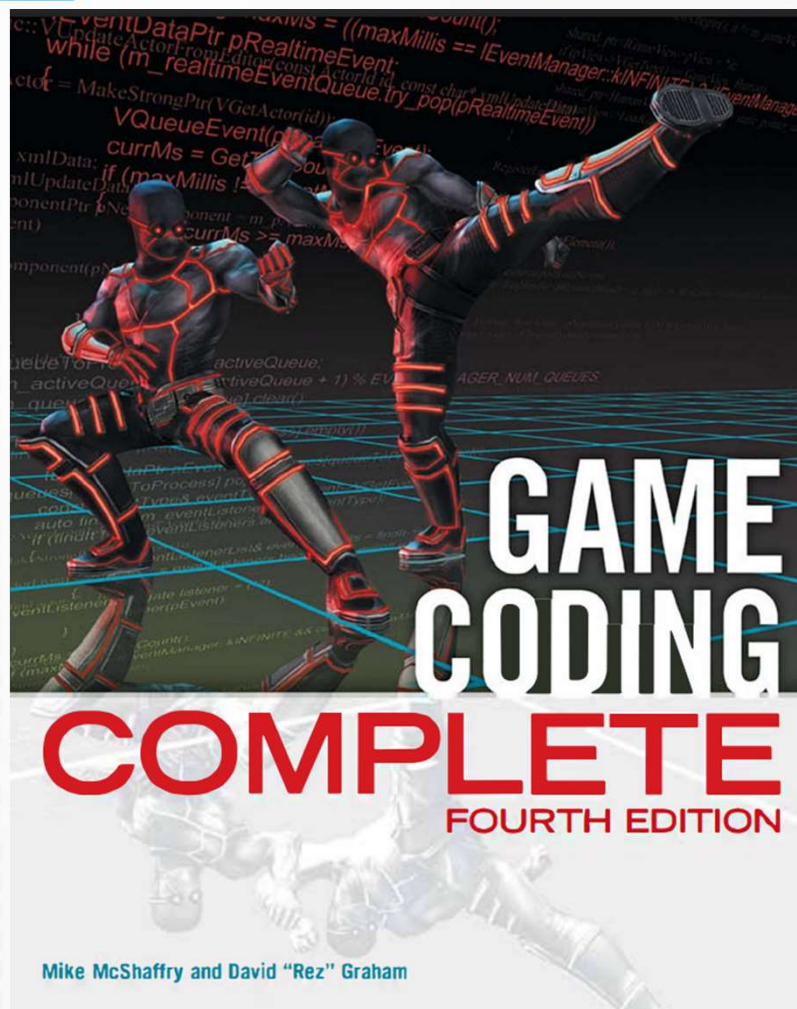
SĐT/Zalo: 037601899

Mục tiêu môn học

- *Biết* các kiến thức cơ bản về lập trình game
- Vận dụng các kĩ thuật cơ bản trong phát triển game
- Khai thác, sử dụng Unity để xử lý các thành phần trong game
- Vận dụng được các kỹ thuật xử lý, quản lý các thành phần trong video game để xây dựng 1 game đơn giản.

Tài liệu tham khảo

- Game coding complete - Mike McShaffry và David "Rez" Graham (Fourth edition, 2013)
- Level up! The guide to great video game design. - Scott Rogers (2010, John Wiley and Sons, Ltd)
- The Art of game design. A book of lenses - Jesse Schell (2008, Elsevier)



TỔNG QUAN VỀ LẬP TRÌNH GAME

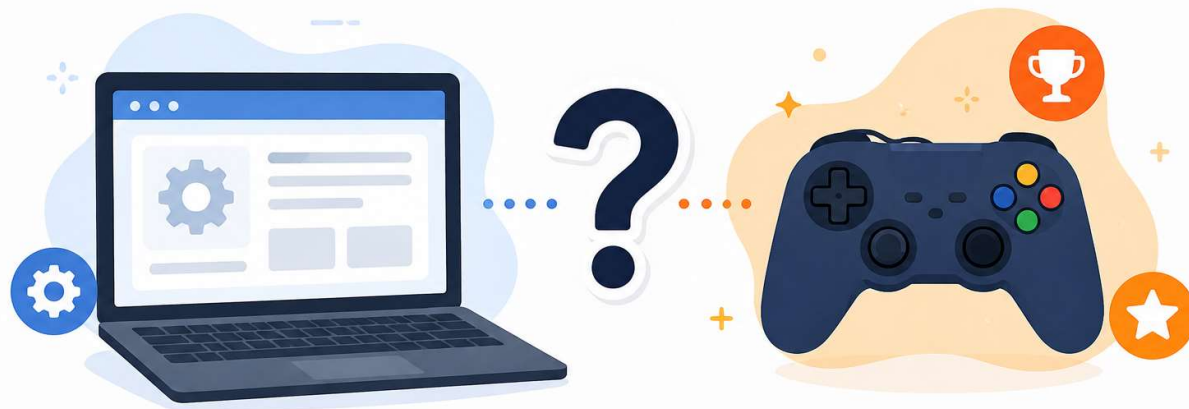
Buổi học đầu tiên sẽ giúp bạn có cái nhìn tổng thể về ngành công nghiệp game, công nghệ, các thành phần của một trò chơi và môi trường thực hành.



TỔNG QUAN NGÀNH CÔNG NGHIỆP GAME

GAME LÀ GÌ?

**Điều gì biến
một phần mềm
thành một trò chơi?**





TRÒ CHƠI = NGƯỜI CHƠI + QUY TẮC + THẾ GIỚI + PHẢN HỒI → TẠO NÊN TRẢI NGHIỆM.

Đó chính là điều biến một phần mềm thành một trò chơi.

GAME KHÔNG CHỈ LÀ CODE

</> CODE (LẬP TRÌNH)

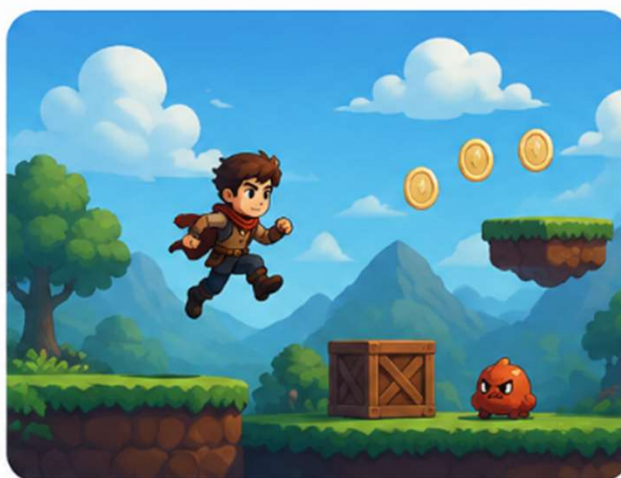
```
// Xử lý di chuyển nhân vật
void Update() {
    float move = Input.GetAxis("Horizontal");
    transform.Translate(Vector3.right *
        move * speed * Time.deltaTime);
}
```

🔊 AUDIO (ÂM THANH)

- 🎵 Nhạc nền
- 🎵 Hiệu ứng âm thanh
- 🗣️ Giọng nhân vật

💡 DESIGN (THIẾT KẾ)

- Cơ chế Gameplay
- Luật chơi
- Cấp độ
- Cân bằng độ khó
- Trải nghiệm người chơi



GAME

= Code + Art + Audio + UI + Design + Data
+ Tools + Process

🎨 ART (ĐỒ HỌA)

🖥️ UI (GIAO DIỆN)

🗄️ DATA (DỮ LIỆU)

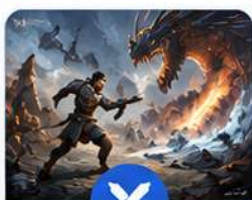
```
{
    "player": {"hp": 100, "speed": 5},
    "enemy": {"hp": 30, "damage": 10},
    "level": 1,
    "coin_value": 1
}
```

Vậy những ai sẽ tạo ra 1 game hoàn chỉnh?



MỘT SỐ THỂ LOẠI GAME PHỔ BIẾN

Mỗi thể loại tập trung vào trải nghiệm và hệ thống gameplay khác nhau, từ đó yêu cầu kỹ thuật và cách thiết kế khác nhau.



Action

Phân xạ, chiến đấu nhanh, nhịp độ cao



Adventure

Khám phá, giải đố, câu chuyện



RPG

Phát triển nhân vật, nhiệm vụ, cốt truyện



Strategy

Quản lý, chiến thuật, ra quyết định



Simulation

Mô phỏng, quản lý thực tế



Racing

Đua xe, tốc độ, điều khiển



Sports

Mô phỏng thể thao, thi đấu



Puzzle

Giải đố, suy luận, tư duy logic



Platformer

Di chuyển qua nền tảng, vượt chướng ngại



Shooter

Bắn súng, chiến đấu, đối kháng

GHI NHỚ

- ✓ Thể loại không quyết định chất lượng game.
- ✓ Một game có thể kết hợp nhiều thể loại (VD: Action + RPG + Adventure).
- ✓ Thể loại ảnh hưởng trực tiếp đến:
 - Gameplay
 - Hệ thống kỹ thuật
 - Nội dung và tài nguyên



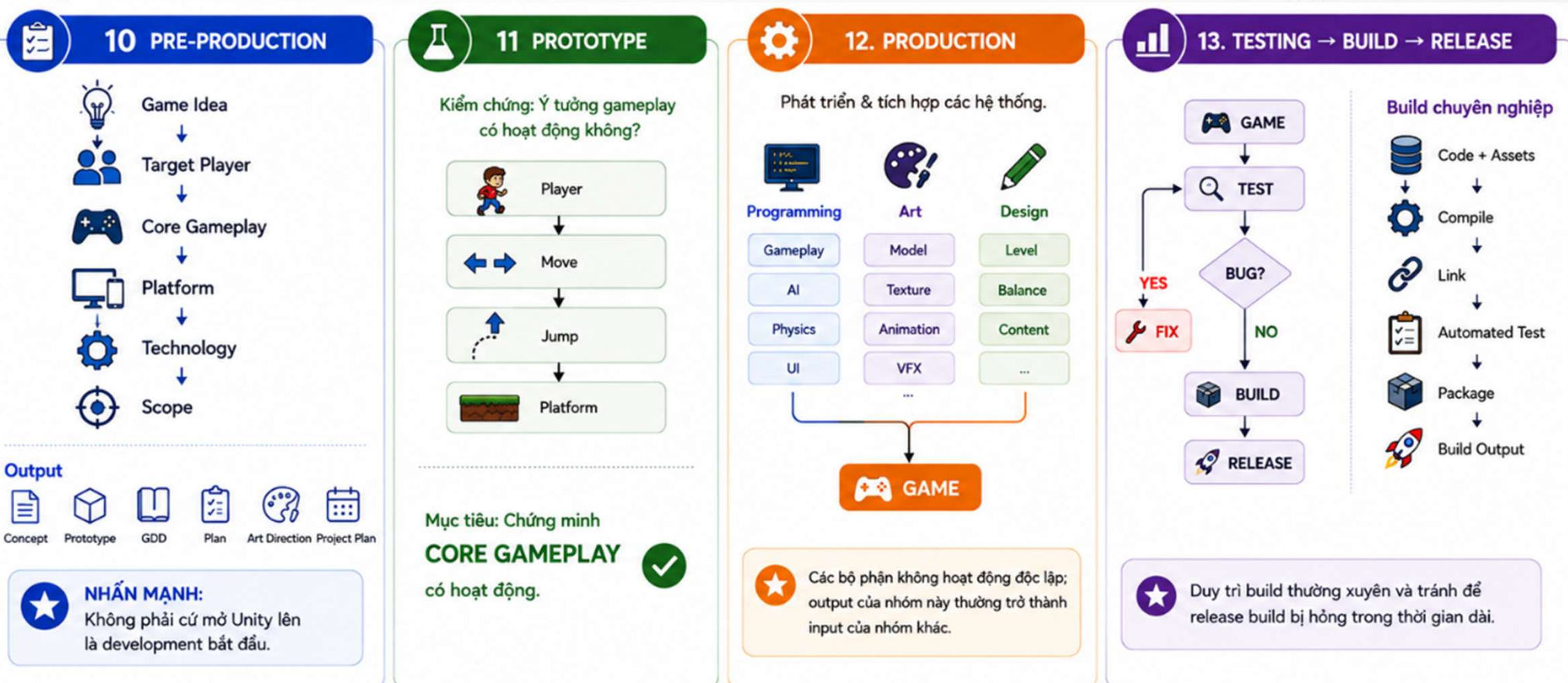
MỤC TIÊU

- ✓ Hiểu sự khác biệt giữa các thể loại game.
- ✓ Liên hệ thể loại với yêu cầu kỹ thuật của game.
- ✓ Làm cơ sở lựa chọn công nghệ phù hợp khi phát triển game.

Quy trình phát triển game



Vòng đời của sản phẩm game



VIDEO GAME PROGRAMMING LÀ GÌ?

I Video Game Programming là quá trình sử dụng phần mềm để hiện thực hóa các hệ thống tạo nên trải nghiệm tương tác của trò chơi.

GAME LÀ GÌ?



Một game là một chương trình mô phỏng thế giới ảo, trong đó người chơi (hoặc AI) tương tác theo các quy tắc của game, để tạo ra trải nghiệm vui vẻ và hấp dẫn.

GAME GỒM NHIỀU HỆ THỐNG PHỐI HỢP VỚI NHAU



Input



Game Logic



Physics /
Collision



AI



Animation



Graphics



Audio



UI



ĐIỂM ĐẶC BIỆT: Rất nhiều hệ thống trong game phải chạy đồng bộ và phản hồi ngay lập tức. Đó chính là sự khác biệt lớn nhất giữa lập trình game và lập trình ứng dụng thông thường.

DÒNG HOẠT ĐỘNG CỦA GAME



Input

Nhận dữ liệu từ người chơi hoặc hệ thống.



Game Logic

Xử lý luật chơi, trạng thái và các hành vi.



Physics / Collision

Tính toán chuyển động và va chạm.



AI

Các tác nhân AI ra quyết định và hành động.



Animation

Cập nhật chuyển động và hoạt ảnh.



Graphics

Hiển thị hình ảnh lên màn hình.



Audio

Phát âm thanh và nhạc nền.



Output

Trình bày trải nghiệm đến người chơi.

LẬP TRÌNH ỨNG DỤNG VS LẬP TRÌNH GAME

LẬP TRÌNH ỨNG DỤNG (APPLICATION PROGRAMMING)

VÍ DỤ: MICROSOFT WORD

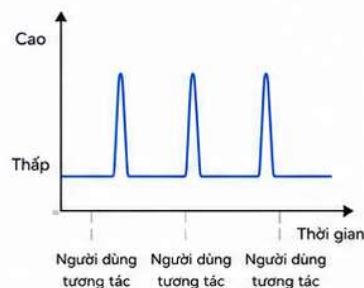


i Nếu người dùng không làm gì thì phần lớn ứng dụng cũng không cần liên tục thay đổi trạng thái.

ĐẶC ĐIỂM CHÍNH

- ✓ Chờ event từ người dùng
- ✓ Xử lý khi có thao tác (click, gõ phím,...)
- ✓ Phần lớn thời gian ở trạng thái idle
- ✓ Tài nguyên hệ thống sử dụng thấp khi không có thao tác
- ✓ Luồng thực thi chủ yếu bị động (event-driven)

MỨC ĐỘ HOẠT ĐỘNG CỦA ỨNG DỤNG



LẬP TRÌNH GAME (GAME PROGRAMMING)

VÍ DỤ: NGƯỜI CHƠI ĐỨNG YÊN TRONG GAME



Người chơi không nhấn phím

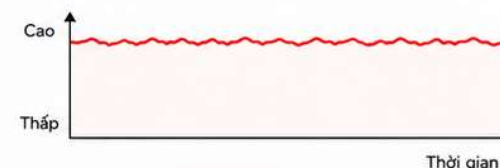


GAME VẪN PHẢI TIẾP TỤC MÔ PHỎNG THẾ GIỚI.

ĐẶC ĐIỂM CHÍNH

- ✓ Luôn chạy liên tục (game loop)
- ✓ Mô phỏng thế giới ảo theo thời gian thực
- ✓ Nhiều hệ thống hoạt động song song
- ✓ Tiêu tốn tài nguyên liên tục (CPU, GPU, RAM,...)
- ✓ Luồng thực thi chủ động (real-time simulation)

MỨC ĐỘ HOẠT ĐỘNG CỦA GAME



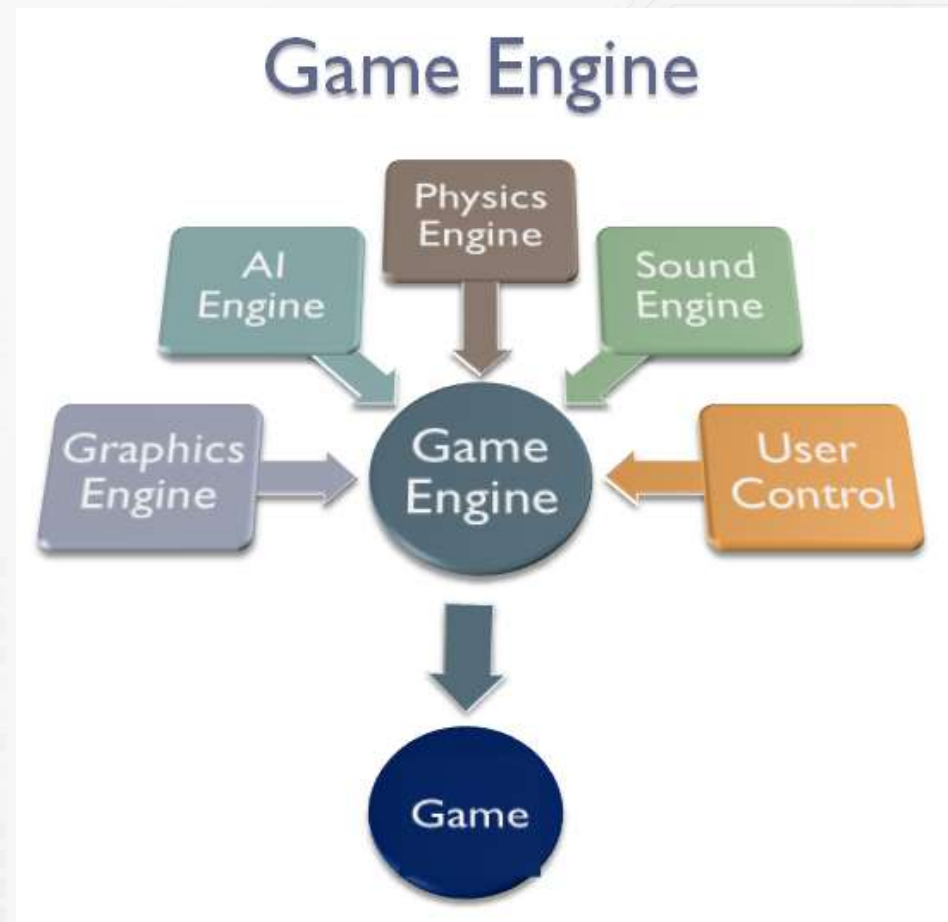
KẾT LUẬN

Ứng dụng như Excel có thể gần như idle khi người dùng không thao tác, trong khi game là một simulation có hoạt động riêng của nó.

Đây là một khác biệt cực kỳ quan trọng.

CÔNG NGHỆ PHÁT TRIỂN GAME

Game Engine là gì ???



Nếu không có Game Engine có thể làm game không ?

✓ CÓ THỂ LÀM ĐƯỢC



Lập trình



Điều khiển



Vật lý



Âm thanh



Quản lý tài nguyên



Logic game



Tự xây dựng từng hệ thống từ đầu.

✗ NHƯNG RẤT KHÓ



Tốn rất nhiều thời gian

Phát triển engine mất vài năm.



Độ phức tạp cao

Cần sâu về đồ họa, toán học, hệ điều hành, tối ưu...



Dễ phát sinh lỗi

Mỗi hệ thống tự viết đều có nguy cơ lỗi.



Khó mở rộng

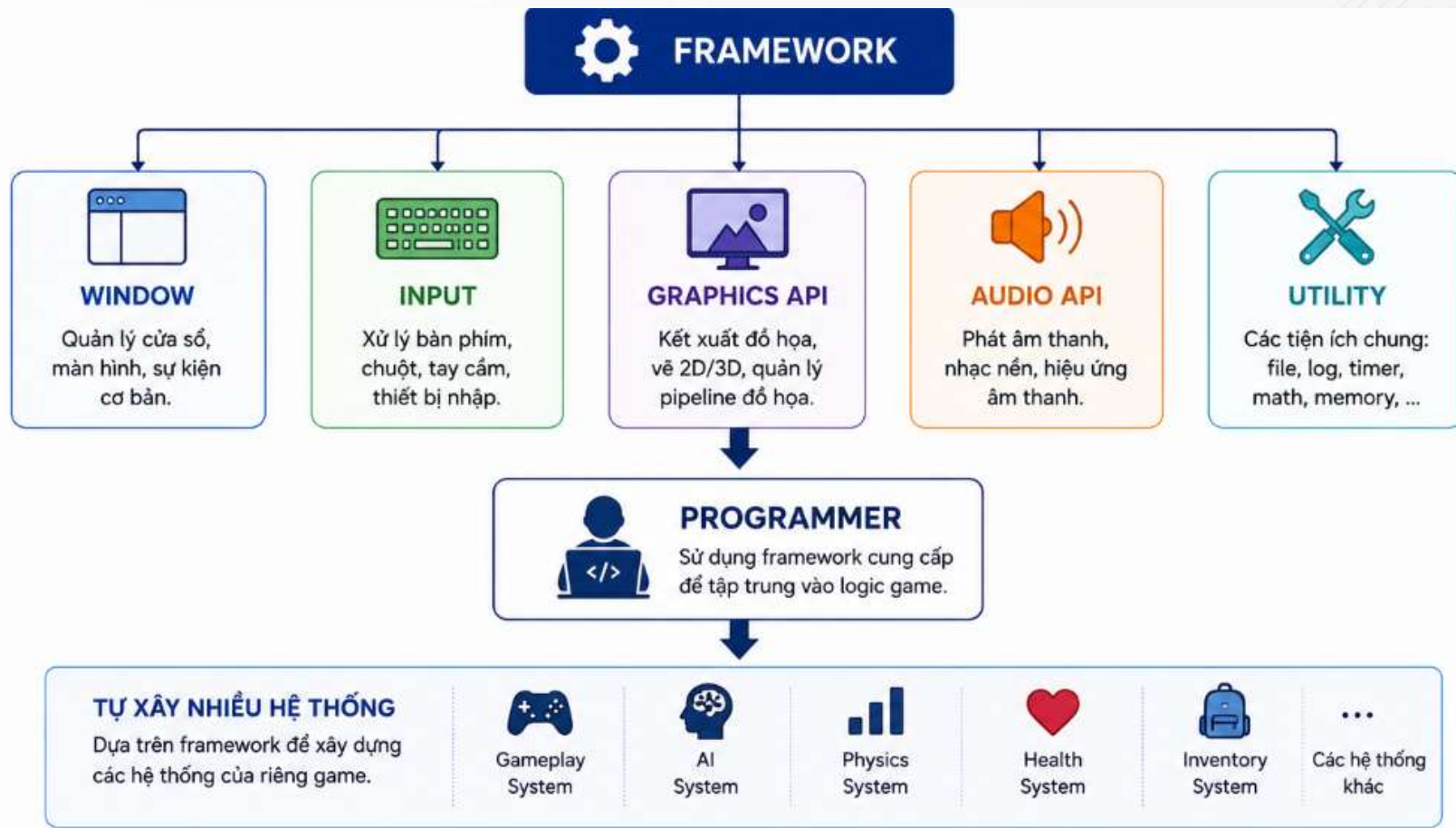
Thêm tính năng mới rất tốn công và dễ ảnh hưởng hệ thống.



Không tập trung vào gameplay

Mất thời gian vào hạ tầng thay vì trải nghiệm người chơi.

Game Framework

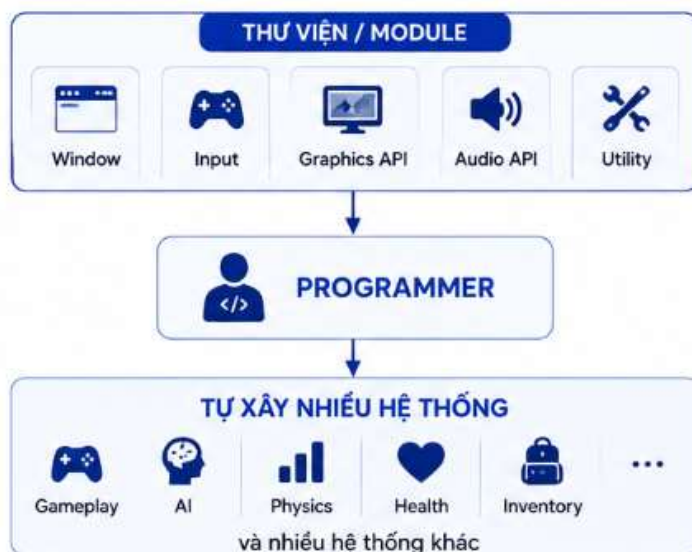


Game Framework vs Game Engine

GAME FRAMEWORK

“Tôi xây game bằng các thư viện”

Bạn tự quyết định cách sử dụng các thành phần.



➔ **Framework** → Bạn tự do xây dựng game theo cách của mình.

VS

GAME ENGINE

“Tôi xây game bên trong một hệ sinh thái”

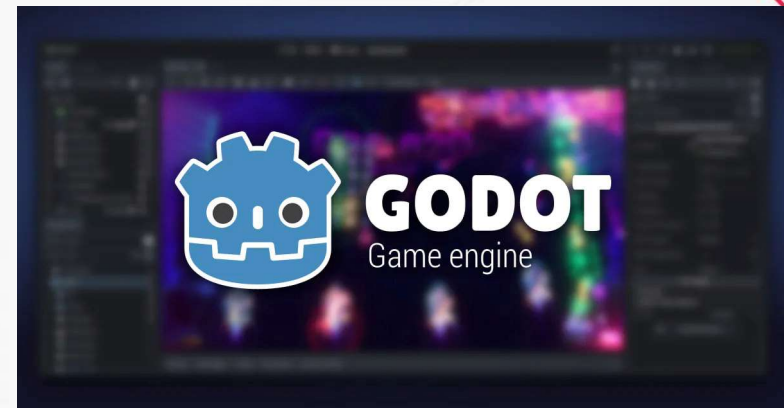
Engine cung cấp đầy đủ công cụ và môi trường phát triển.



➔ **Engine** → Bạn tập trung vào nội dung và gameplay, engine lo phần còn lại.

THIẾT KẾ VÀ PHÁT TRIỂN GAME

Các engine phổ biến



THIẾT KẾ VÀ PHÁT TRIỂN GAME

Lựa chọn engine tốt nhất

CÂU HỎI KHÔNG PHẢI

Engine nào
tốt nhất? ?



CÂU HỎI ĐÚNG

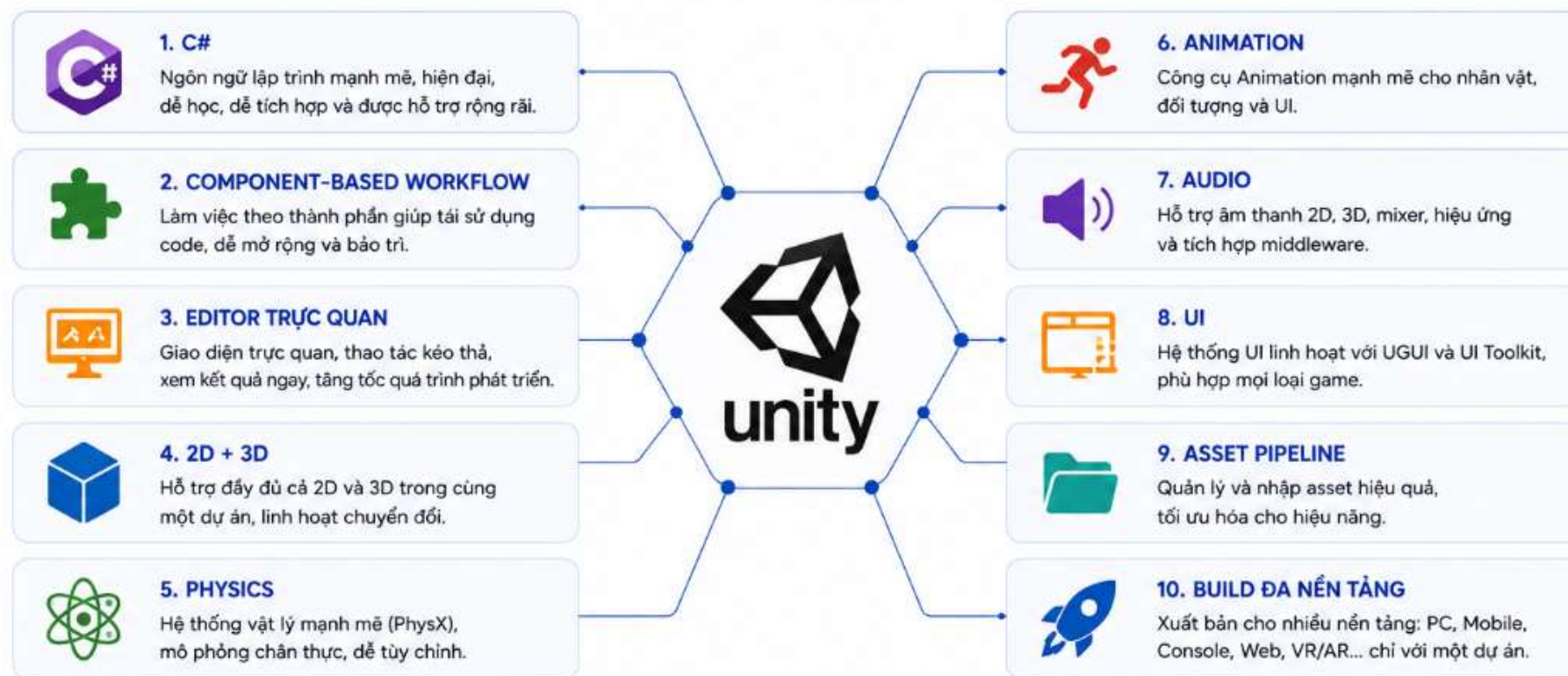
Engine nào **phù hợp nhất**
với **game** và **đội phát triển**?



Không có engine tốt nhất cho mọi game.
Hãy chọn engine **phù hợp nhất** để đội của bạn làm ra **game tốt nhất!**



Vì sao trong môn học dạy Unity



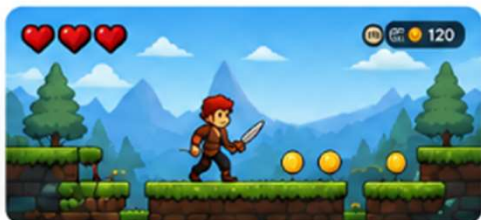
CÁC THÀNH PHẦN CỦA MỘT TRÒ CHƠI



1. GAMEPLAY

Là linh hồn của trò chơi.

Bao gồm luật chơi, cơ chế, mục tiêu, thử thách, hệ thống phần thưởng và vòng lặp trải nghiệm.



2. GIAO DIỆN - ÂM THANH

Bao gồm các yếu tố giúp game trở nên sinh động, dễ tiếp cận và tạo cảm xúc cho người chơi.



GIAO DIỆN (UI/UX)

Cầu nối giữa người chơi và trò chơi. Bao gồm HUD, menu, thông báo, hướng dẫn và trải nghiệm người dùng.



HIỆU ỨNG HÌNH ẢNH

Các hiệu ứng kỹ năng, va chạm, ánh sáng, chuyển động giúp tăng tính hấp dẫn.



ÂM THANH

Tăng cường cảm xúc và sự nhập vai thông qua nhạc nền, hiệu ứng âm thanh, lời thoại.



3. DỮ LIỆU VÀ TÀI NGUYÊN

Lưu trữ và quản lý toàn bộ dữ liệu và tài nguyên của trò chơi: level, nhân vật, vật phẩm, cấu hình, kịch bản, save game,...



Scenes



Characters



Items



Audio



Scripts



Textures



Models



Save Data

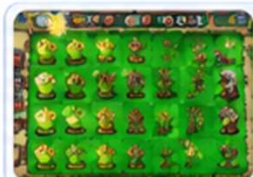
Chúng ta hiểu Gameplay như thế nào?

MÔ HÌNH GAMEPLAY CƠ BẢN



Ý NGHĨA: Một gameplay hấp dẫn là gameplay tạo ra vòng lặp rõ ràng, phần thưởng có ý nghĩa và nhịp điệu phù hợp với trải nghiệm mong muốn.

VÍ DỤ CÁC MÔ HÌNH GAMEPLAY PHỔ BIẾN



1. TOWER DEFENSE

Xây phòng thủ → Kẻ địch xuất hiện
→ Phòng thủ tiêu diệt → Nhận tài nguyên
→ Nâng cấp → Kẻ địch mạnh hơn

Ví dụ game

Plants vs. Zombies
Kingdom Rush



2. ENDLESS RUNNER

Chạy → Né chướng ngại vật → Thu thập
vật phẩm → Đi xa hơn → Mở khóa
nhân vật / nâng cấp → Chạy tiếp

Ví dụ game

Subway Surfers
Temple Run

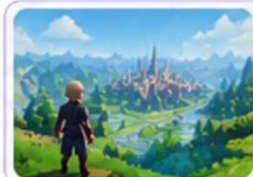


3. MATCH-3 PUZZLE

Ghép 3+ → Hoàn thành mục tiêu →
Nhận điểm / sao → Vượt màn →
Màn khó hơn

Ví dụ game

Candy Crush Saga
Homescapes



4. OPEN WORLD RPG

Khám phá → Nhiệm vụ → Chiến đấu →
Nhận phần thưởng → Nâng cấp nhân vật →
Mở khu vực / nội dung mới

Ví dụ game

Genshin Impact
The Legend of Zelda:
Breath of the Wild

THIẾT KẾ VÀ PHÁT TRIỂN GAME

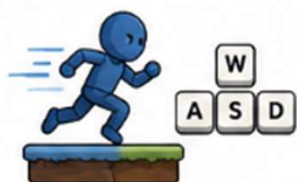
Vậy lập trình Gameplay sẽ phải làm những gì ?



Gameplay Programming

CÁC THÀNH PHẦN CHÍNH TRONG GAMEPLAY PROGRAMMING

1. MOVEMENT



Điều khiển nhân vật di chuyển, nhảy, né, trèo, lăn... và xử lý vật lý liên quan.

2. COMBAT



Xử lý tấn công, phòng thủ, sát thương, phạm vi ảnh hưởng, hệ thống máu và hồi máu.

3. INTERACTION



Tương tác với NPC, vật thể, công tắc, cửa, nhiệm vụ và các đối tượng trong game.

4. ITEMS



Quản lý vật phẩm: nhặt, sử dụng, trang bị, tiêu hao và hiệu ứng liên quan.

5. ABILITIES



Hệ thống kỹ năng và phép: kích hoạt, hồi chiêu, tiêu hao năng lượng, hiệu ứng.

6. SCORE



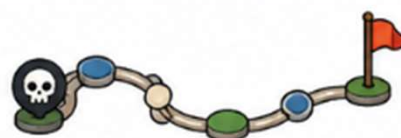
Tính điểm dựa trên hành động của người chơi và hiển thị kết quả trong game.

7. GAME RULES



Các luật chơi tổng quát chi phối toàn bộ game: mục tiêu, điều kiện, giới hạn và cơ chế cốt lõi.

8. LEVEL RULES



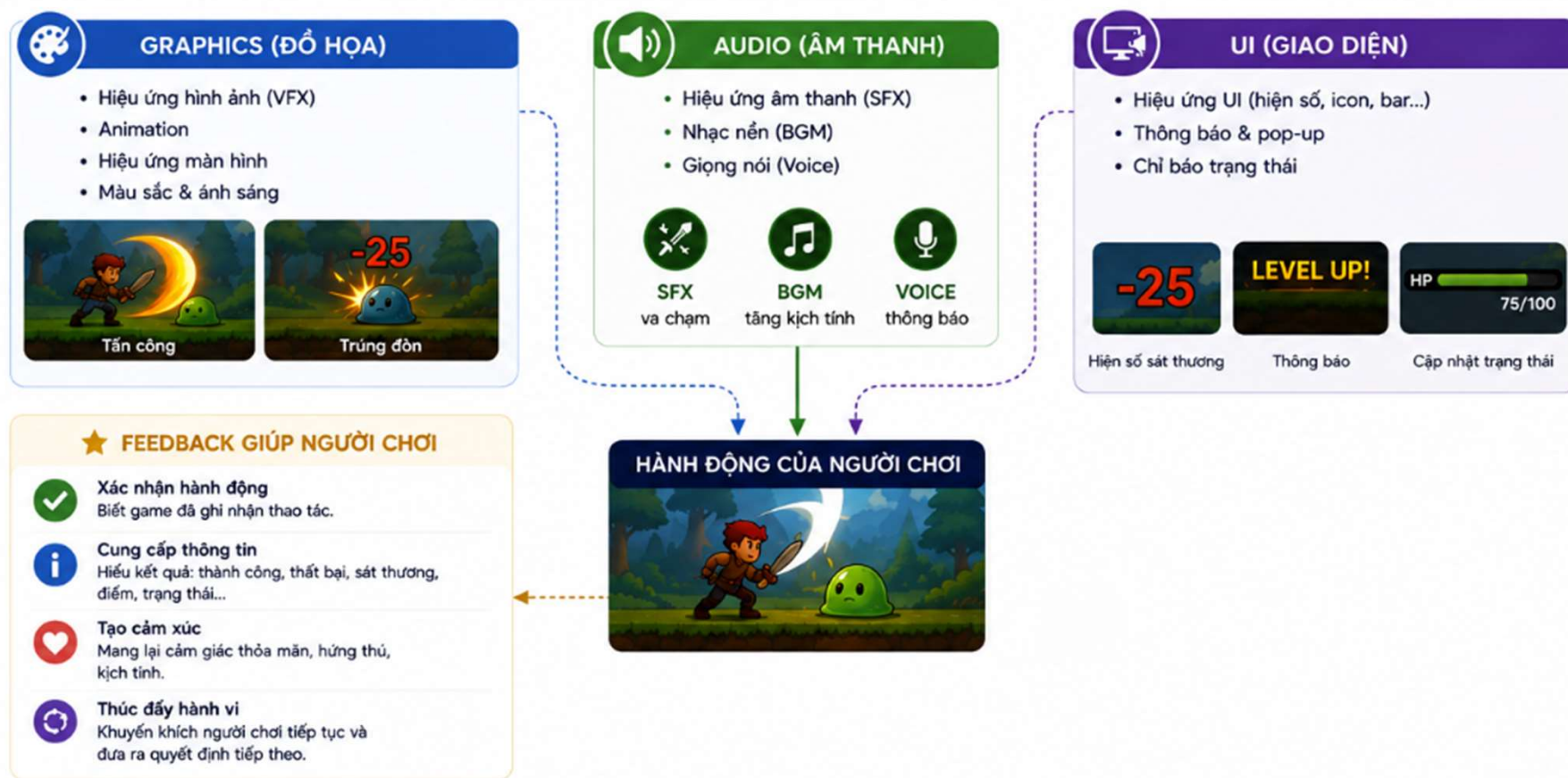
Luật riêng của từng màn/level: mục tiêu mức độ, giới hạn thời gian, kẻ địch, điều kiện đặc biệt...

9. WIN / LOSE



Xác định điều kiện thắng / thua, hiển thị kết quả và chuyển đổi trạng thái game.

Đồ họa, âm thanh và giao diện



Dữ liệu trong game



GAME DATA

Dữ liệu cấu trúc điều khiển và lưu trữ trạng thái game.



Dữ liệu Gameplay

Chỉ số nhân vật, máu, năng lượng, điểm số, cấp độ, kỹ năng...



Dữ liệu Quy tắc & Hệ thống

Luật chơi, điều kiện thắng/thua, cơ chế AI, xác suất rơi đồ...



Dữ liệu Tiến trình

Nhiệm vụ đã hoàn thành, thành tựu, cấp độ đã mở, lưu game...



Dữ liệu Người chơi

Tùy chọn, cài đặt, hồ sơ người chơi, xếp hạng, lịch sử...



Dữ liệu Cấu hình

Thiết lập độ khó, cân bằng game, tham số hệ thống...

Ví dụ lưu trữ

```
{
  "player": {
    "name": "Hero",
    "level": 15,
    "hp": 120/120,
    "exp": 4500
  },
  "inventory": [
    { "id": "sword_001", "qty": 1 },
    { "id": "potion_hp", "qty": 5 }
  ],
  "quest": {
    "chapter": 3,
    "progress": 65
  },
  "settings": {
    "volume": 80,
    "language": "vi"
  }
}
```



Game Data thường ở dạng số liệu, văn bản, bảng... được lưu trữ trong file (JSON, CSV, XML, DB...) hoặc hệ thống lưu trữ.



GAME ASSETS

Tài nguyên tạo nên nội dung và hình ảnh trong game.



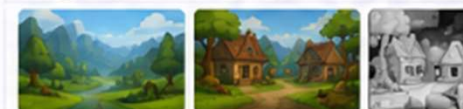
Nhân vật (Character)

Model 2D/3D, trang phục, biểu cảm, rig, skin...



Môi trường (Environment)

Địa hình, cây cối, kiến trúc, ánh sáng, bầu trời...



Vật phẩm (Items)

Vũ khí, giáp, vật phẩm, đạo cụ, rương, tiền...



Hiệu ứng (Effects)

Hiệu ứng kỹ năng, ánh sáng, khói, lửa, va chạm...



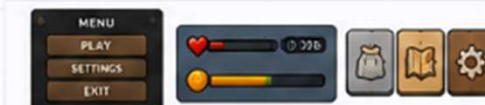
Âm thanh (Audio)

Nhạc nền, hiệu ứng âm thanh, giọng nói, ambient...



Giao diện (UI/UX)

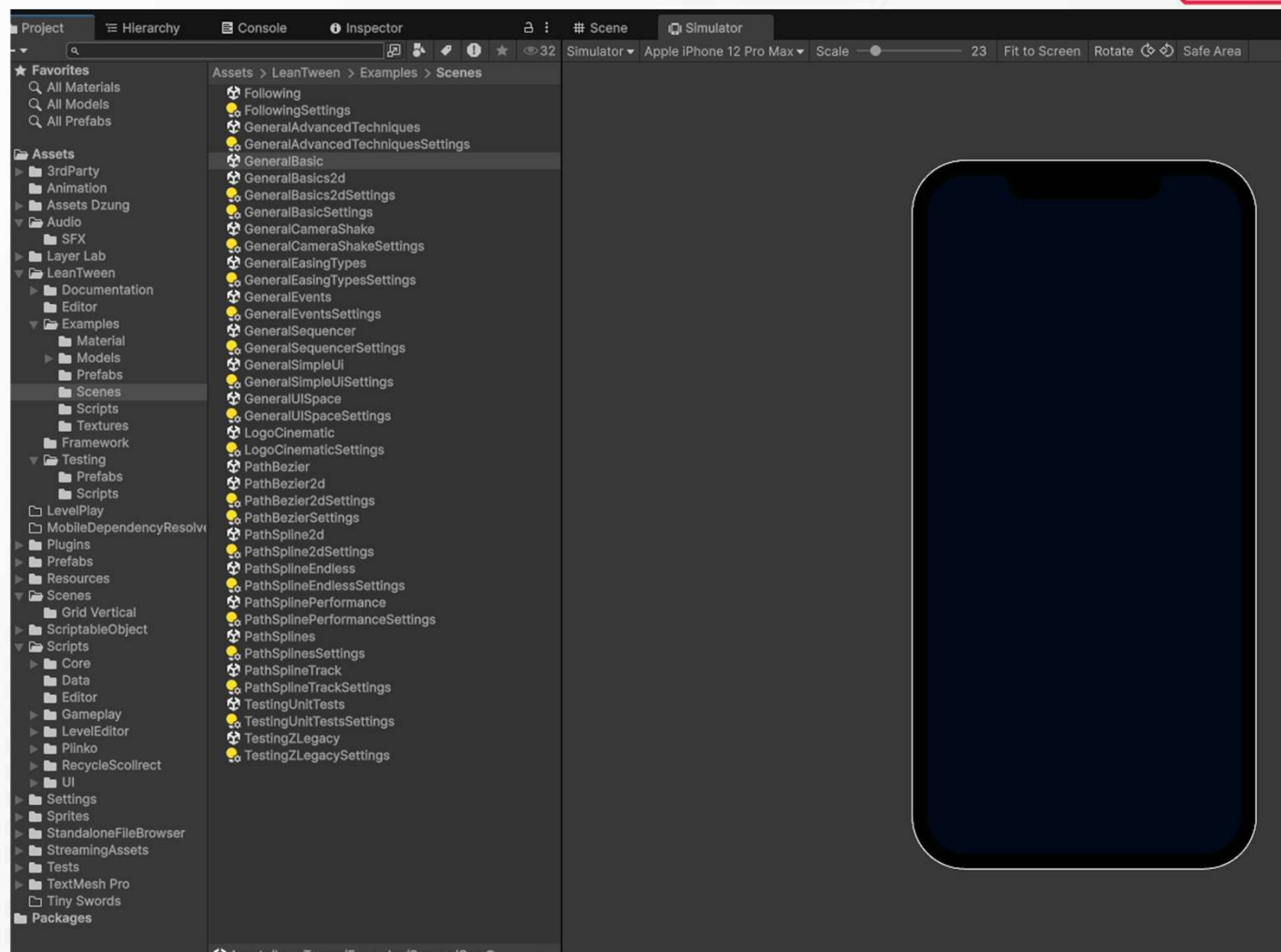
Menu, HUD, nút bấm, icon, font chữ, tooltip...



Game Assets là các tệp tin (texture, model, audio, animation, font...) được import và quản lý trong engine (ví dụ: Unity Asset Database).

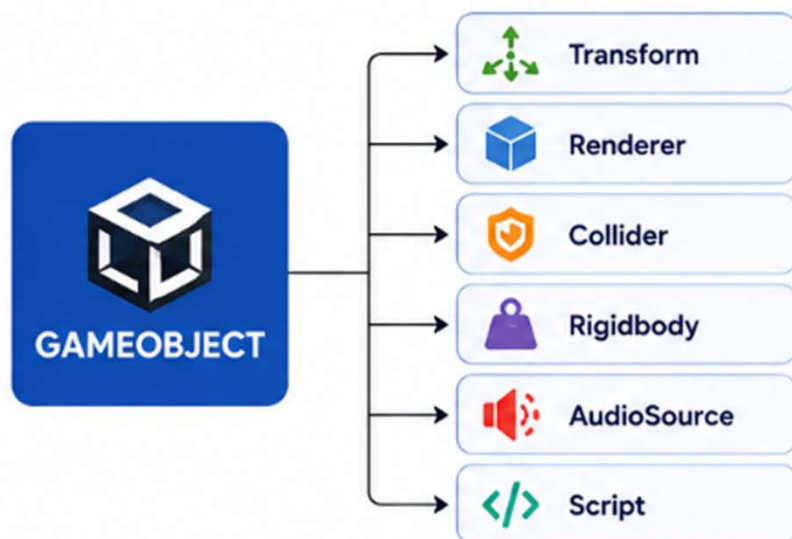
GIỚI THIỆU MÔI TRƯỜNG THỰC HÀNH LẬP TRÌNH GAME CƠ BẢN

Unity Editor



TƯ DUY UNITY

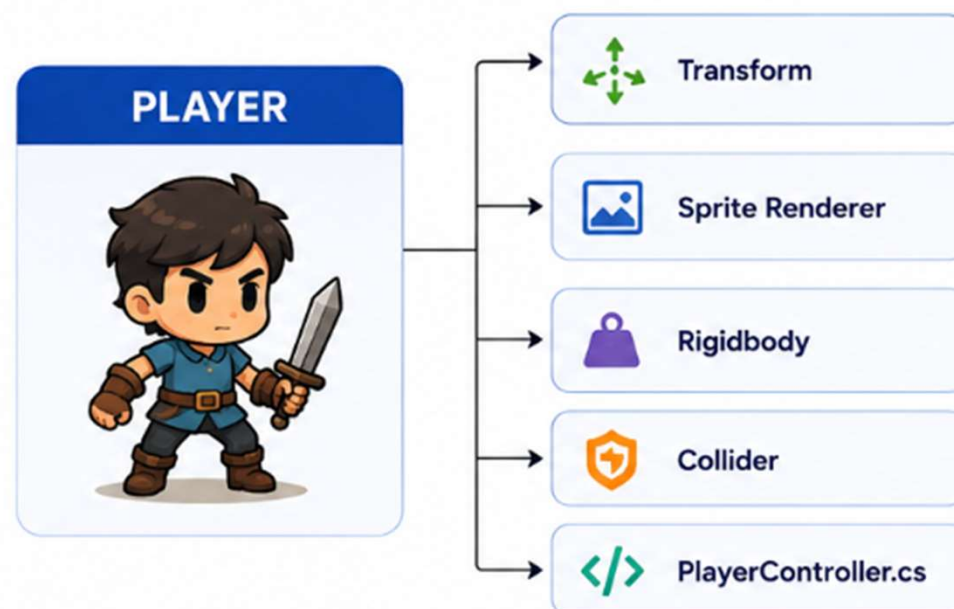
- Một trong những khái niệm đầu tiên:



 GameObject là “**vật chứa**”.

 Component cung cấp **khả năng**.

- Ví dụ:



CẤU TRÚC PROJECT GAME CHUẨN

Ý nghĩa từng thành phần trong project Unity

Name	Date modified	Type
 Assets	7/26/2026 10:27 PM	File folder
 Docs	7/26/2026 9:54 PM	File folder
 Library	7/27/2026 1:04 AM	File folder
 Logs	7/26/2026 10:22 PM	File folder
 Packages	7/26/2026 9:54 PM	File folder
 ProjectSettings	7/26/2026 10:27 PM	File folder
 UserSettings	7/26/2026 10:27 PM	File folder



GHI NHỚ QUAN TRỌNG

- ✓ Chỉ cần chia sẻ/commit các thư mục: Assets, Docs, Packages, ProjectSettings.
- ✓ Không commit các thư mục: Library, Logs, UserSettings.
- ✗ Luôn giữ cấu trúc thư mục gọn gàng, rõ ràng để dễ quản lý và mở rộng dự án.
- ✓ Đặt tên file, folder theo quy tắc thống nhất của team.

1. Assets

Nơi lưu trữ toàn bộ tài nguyên của game.

- Script, Prefab, Scene, Model, Texture, Audio, Animation,...
- Đây là thư mục quan trọng nhất, cần được quản lý khoa học.

2. Docs

Lưu trữ tài liệu dự án.

- GDD, thiết kế game, kế hoạch, hướng dẫn, tài liệu kỹ thuật,...

3. Library

Thư mục do Unity tự động tạo.

- Lưu các file tạm, cache, dữ liệu biên dịch của Unity.
- KHÔNG chỉnh sửa thủ công hoặc commit lên Git.

4. Logs

Lưu log hệ thống và log lỗi trong quá trình chạy Unity.

- Hỗ trợ debug và kiểm tra lỗi.
- Có thể xóa khi không cần thiết.

5. Packages

Quản lý các gói (package) được sử dụng trong dự án.

- Các gói từ Unity Registry hoặc custom package.
- Giúp dễ dàng quản lý và chia sẻ dependencies.

6. ProjectSettings

Chứa các thiết lập cấu hình của dự án.

- Input, Graphics, Physics, Quality, Editor, Tag, Layer,...
- Tùy chỉnh cách Unity hoạt động với dự án của bạn.

7. UserSettings

Lưu các thiết lập cá nhân của từng người dùng.

- Layout, màu giao diện, cấu hình Editor riêng...
- KHÔNG nên commit lên Git.

Assets Folder



UNITY PROJECT

Dự án Unity bao gồm toàn bộ nội dung và cấu hình cần thiết để phát triển game.



Các thành phần chính cấu thành dự án trước khi build.

BUILD

Unity sẽ biên dịch, tối ưu và đóng gói project thành file thực thi hoặc gói cài đặt.

TARGET PLATFORM

Chọn nền tảng mục tiêu mà bạn muốn xuất bản game.

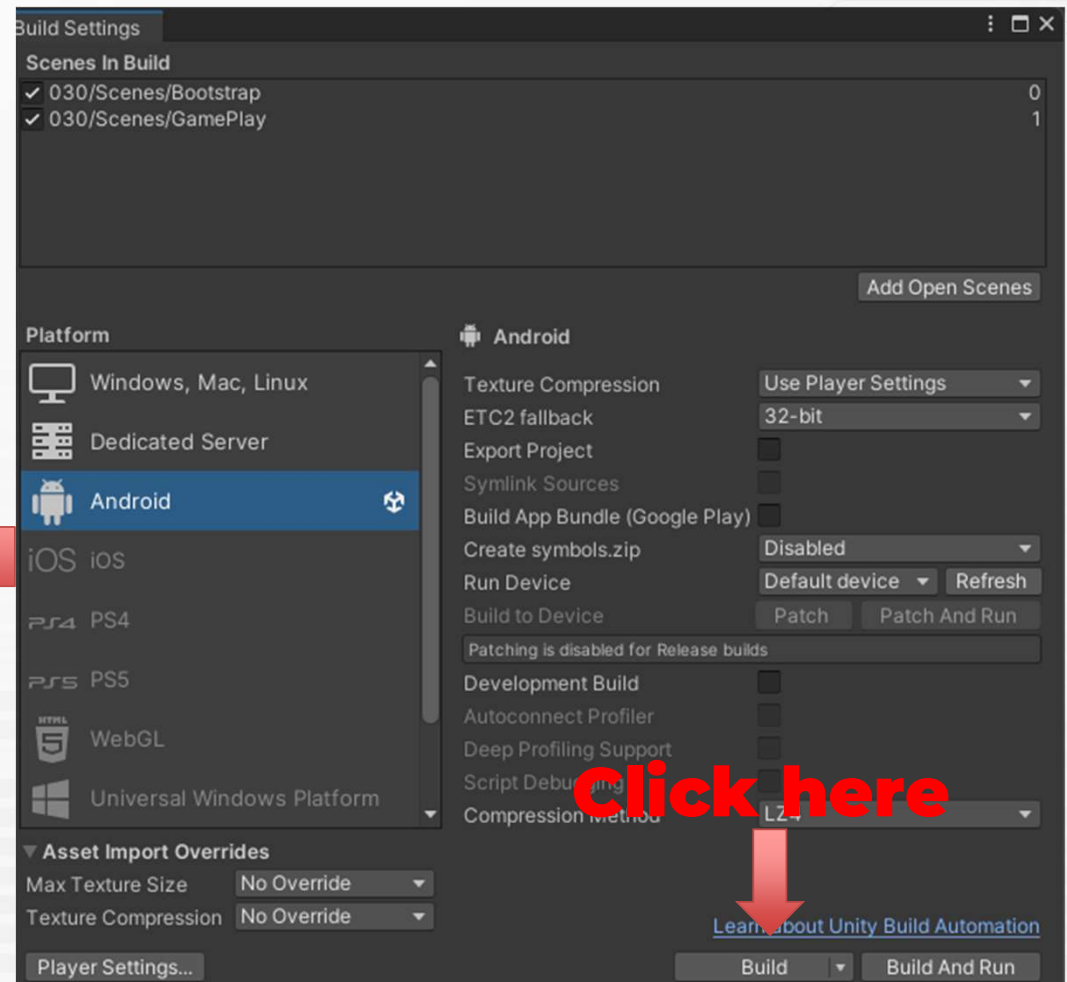


Xuất bản ra nền tảng tương ứng để người dùng có thể cài đặt hoặc chơi trực tiếp.

Chu trình Build Game

Thao tác build game

Chọn Platform



THIẾT KẾ VÀ PHÁT TRIỂN GAME

Tổng kết

1. MỤC TIÊU BUỔI HỌC



Hiểu cấu trúc một game

Các thành phần cơ bản tạo nên một game.



Làm quen với Unity

Giao diện, cách tạo project và các thao tác cơ bản.



Quản lý dự án hiệu quả

Cấu trúc thư mục chuẩn, quản lý asset khoa học.



Hiểu quy trình phát triển

Từ ý tưởng đến sản phẩm và phát hành game.

2. NỘI DUNG CHÍNH

1

Tổng quan về lập trình game

Game là gì? Thành phần của một game.
Vai trò của lập trình trong phát triển game.

2

Giới thiệu Unity

Unity là gì? Ưu điểm.
Giao diện Unity, các cửa sổ chính.

3

Cấu trúc dự án Unity

Cấu trúc thư mục chuẩn.
Quản lý Assets, Scenes, Scripts, ...

4

Quy trình phát triển & Build

Quy trình làm việc với Unity.
Build và xuất bản ra nền tảng.

5

Thực hành

Tạo project mới.
Tạo Scene, GameObject đơn giản.
Lưu và chạy thử game.

3. KẾT QUẢ BUỔI HỌC



Nắm được tổng quan về lập trình game.



Hiểu cấu trúc cơ bản của dự án Unity.



Tạo được project mới và Scene đơn giản.



Hiểu quy trình Build game và các nền tảng xuất bản.



Sẵn sàng cho các buổi học tiếp theo.



VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG CDIT